Breakout Lab - Conversation Log

Tool activity is summarized in grey; messages are shown as written.

USER

Fix ball brick collision mentioned in task 1 of README.md

```
[tools] Read README.md; listed repo files (found sources under breakout/game/); read game_engine.py
and brick.py
```

```
[tools] Edited breakout/game/game_engine.py: remove a brick when its hits_remaining reaches 0
```

CLAUDE

I fixed Task 1 in game_engine.py:65-69. When a brick takes a hit and hits_remaining reaches 0, it is now removed from self.bricks. I haven't run the game to check it.

Removing from the list inside the loop is safe because the loop breaks right after. The "Bricks left" counter will now drop as bricks break.

The README says the bug is in game/game_engine.py, but the files are under breakout/game/. The comment in game_engine.py also mentions game/collision.py, but the missing check was in update(). I didn't change collision.py.

USER

Now implement lives (task 2)

```
[tools] Read renderer.py and main.py; edited game_engine.py (STARTING_LIVES, lives, game_over,
restart on R, lives display, game-over banner); updated main.py docstring; verified logic with a
stubbed pygame (pygame not installed)
```

CLAUDE

Task 2 is implemented in game_engine.py. I checked the logic with a stubbed pygame, since pygame isn't installed here, so I haven't run the actual game or looked at the display.

- Lives: the engine starts with 3 lives (STARTING_LIVES). Each time the ball falls below the paddle, you lose one and the ball resets to its start position.
- Game over: when the last life is lost, update() stops and the screen shows "GAME OVER - press R to restart".
- Restart: pressing R after game over starts a fresh game with 3 lives and a full set of bricks.
- Display: "Lives: N" appears in the top-right corner.
- Check: in the stubbed test, losing 3 balls gave 0 lives and game over, and restarting gave 3 lives and no game over.

I also updated the controls line in the docstring of main.py to mention R.

USER

It works. Now task 3.

```
[tools] Rewrote brick.py (kinds, colors, hit()); updated game_engine.py (brick layout, hit(),
breakable-only counter); tested brick logic with stubbed pygame
```

CLAUDE

Task 3 is implemented, but I haven't run the game itself. I checked the brick logic with a stubbed pygame: a strong brick takes 3 hits to break, an unbreakable one survives any number of hits, and a normal one breaks on the first hit.

- Brick types (brick.py): Brick now takes a kind of normal, strong or unbreakable. Normal bricks are red and break after 1 hit. Strong bricks are blue and take 3 hits; they get lighter as they're damaged. Unbreakable bricks are grey and are never destroyed.

- Hit handling: a new brick.hit() method does the hit tracking and returns whether the brick is destroyed. The ball still bounces off unbreakable bricks.
- Layout (game_engine.py): the top row is strong, the rest are normal, and there are two unbreakable bricks at the left and right ends of the third row. You can change this in _brick_kind.
- Counter: "Bricks left" now counts only breakable bricks. Otherwise it could never reach 0.

I changed the Brick constructor: the old hits_remaining and color arguments are gone, and both are now derived from kind.

USER

Alright, start task 4.

```
[tools] Edited game_engine.py (score, multiplier, BRICK_POINTS, MAX_MULTIPLIER, reset on miss, score display); tested with a stubbed pygame
```

CLAUDE

Task 4 is implemented in game_engine.py. I haven't run the game, and I only checked the logic with a stubbed pygame.

- Scoring: each brick you destroy is worth 10 points times the current multiplier.
- Multiplier: it starts at x1 and goes up by 1 for each brick destroyed in a row, capped at x8 (MAX_MULTIPLIER).
- Combo reset: losing the ball resets the multiplier to x1. Bouncing off the paddle doesn't reset it, since the README says the combo only breaks when you miss the ball.
- Strong bricks: hits that don't destroy a brick score nothing and don't change the combo. Hitting an unbreakable brick does nothing to the combo either.
- Display: "Score: N xM" is shown under the "Bricks left" counter.
- Restart: pressing R after game over resets the score and multiplier.

In the stubbed test, three bricks in a row scored 60 and raised the multiplier to x4. A fourth brick scored 40 and the multiplier dropped back to x1 on the miss. That fourth brick was an artifact of my stub still reporting a collision on the miss frame.

USER

The score text overlaps with the bricks on the top left, move the bricks down by about 20 pixels.

```
[tools] Edited game_engine.py: BRICK_TOP_MARGIN 50 -> 70
```

CLAUDE

I moved the bricks down 20 pixels by changing BRICK_TOP_MARGIN from 50 to 70 in game_engine.py:24. The score text should now clear the top row. I haven't run the game to confirm.

The lowest row of bricks is now 20 pixels closer to the paddle. There's still plenty of room above it for the ball.

USER

Update the README now to reflect that the tasks are completed.

```
[tools] Edited README.md: controls line, intro, and a Done marker plus status note for each of Tasks 1-4
```

CLAUDE

I updated README.md to show all four tasks as done.

- Task headings: each one now has a Done marker and a short status note describing what was implemented.
- Intro: it now says the bug and three features were completed.
- Controls: the line now mentions R to restart.

I left the Submission Checklist alone, since the before and after videos and the chat link are yours to do. The Folder Structure section still lists the files without the breakout/ prefix they actually sit under. I can fix that if you want.

USER

Can you turn this entire conversation into a pdf?